

## **IMPLEMENTATION PSEUDO CODE:**

### **Main application:**

BEGIN

Launch Aurelia University Campus Parking & Mobility Office System

Initialize UI theme and application components

Display Login Screen

Authenticate User

IF authentication fails THEN

    Display authentication error

    Return to Login

END IF

Create User Session

Display Dashboard

WHILE application is running

    User selects a module

    CASE selected module OF

        USER MANAGEMENT:

            Perform user operations

        VEHICLE MANAGEMENT:

            Perform vehicle operations

        PARKING ZONES:

            Manage parking zones

        PARKING SLOTS:

            Manage parking slots

        SLOT AVAILABILITY:

            Display current slot states

        SMART RECOMMENDATION:

            Evaluate eligible slots

            Rank candidates

Recommend best slot

Explain recommendation

#### RESERVATIONS:

Validate availability

Check conflicts

Create/cancel reservation

#### VEHICLE ENTRY:

Validate vehicle

Create active session

Occupy slot

#### VEHICLE EXIT:

Find active session

Calculate duration

Calculate fee

Complete session

Release slot

#### PAYMENTS:

Process payment

Update payment status

#### PARKING PASSES:

Create/manage parking passes

#### VIOLATIONS:

Record/manage violations

#### PARKING HISTORY:

Display historical sessions

#### DASHBOARD:

Calculate and display current metrics

#### REPORTS:

Generate requested report

#### LOGOUT:

Clear user session

Return to Login

```
END CASE
END WHILE
Close application
END
```

## IMPLEMENTATION CODE: Reservation & Slot Allocation

```
package com.aurelia.parking.service; import
java.sql.Connection; import
java.sql.PreparedStatement; import
java.sql.ResultSet; import java.sql.SQLException;
import com.aurelia.parking.database.DBConnection;
public class ParkingService {
    /*
     * Creates a reservation and changes the selected
     * parking slot status to RESERVED.
     */
    public boolean createReservation(
        int userId,
int vehicleId,
int slotId,
        String reservationDate,
        String startTime,
        String endTime) {
        Connection connection = null;
        String checkSlot =
            "SELECT status FROM parking_slots WHERE slot_id = ?";
        String checkConflict =
            "SELECT COUNT(*) FROM reservations " +
            "WHERE slot_id = ? " +
            "AND reservation_date = ? " +
            "AND status = 'ACTIVE' " +
            "AND start_time < ? " +
            "AND end_time > ?";
```

---

```

String insertReservation =
    "INSERT INTO reservations " +
    "(user_id, vehicle_id, slot_id, reservation_date, " +
    "start_time, end_time, status) " +
    "VALUES (?, ?, ?, ?, ?, ?, 'ACTIVE)";

String updateSlot =
    "UPDATE parking_slots " +
    "SET status = 'RESERVED' " +
    "WHERE slot_id = ?";    try {
    connection = DBConnection.getConnection();

    // Start transaction
    connection.setAutoCommit(false);    /* Step 1:
    Check slot status */    try (PreparedStatement ps =
    connection.prepareStatement(checkSlot)) {
    ps.setInt(1, slotId);

    ResultSet rs = ps.executeQuery();
    if (!rs.next()) {
    connection.rollback();    return
    false;

    }

    String status = rs.getString("status");
    if (!"AVAILABLE".equals(status)) {
    connection.rollback();    return
    false;

    }
    }

    /* Step 2: Check reservation conflict */    try
    (PreparedStatement ps =
    connection.prepareStatement(checkConflict)) {
    ps.setInt(1, slotId);    ps.setString(2, reservationDate);
    ps.setString(3, endTime);    ps.setString(4, startTime);
    ResultSet rs = ps.executeQuery();    if (rs.next() &&

```

---

```

rs.getInt(1) > 0) {                connection.rollback();
return false;
    }
}

/* Step 3: Create reservation */    try
(PreparedStatement ps =
connection.prepareStatement(insertReservation)) {
ps.setInt(1, userId);              ps.setInt(2, vehicleId);
ps.setInt(3, slotId);              ps.setString(4, reservationDate);
ps.setString(5, startTime);        ps.setString(6, endTime);
ps.executeUpdate();
    }

/* Step 4: Update parking slot */    try
(PreparedStatement ps =
connection.prepareStatement(updateSlot)) {
ps.setInt(1, slotId);              ps.executeUpdate();
    }

// Commit both operations
connection.commit();               return
true;

    } catch (SQLException e) {
        try {                if
(connection != null) {
connection.rollback();
        }

        } catch (SQLException rollbackError) {
            System.err.println(
                "Rollback failed: " + rollbackError.getMessage()
            );                }                return false;
    } finally {                try {                if
(connection != null) {
connection.setAutoCommit(true);
connection.close();

```

```

    }
} catch (SQLException e) {
    System.err.println(
        "Connection closing failed: " + e.getMessage()
    );
}
}
}
}
}
}

```

## IMPLEMENTATION CODE:

### Database Connection Code

```

package com.aurelia.parking.database;
import java.sql.Connection; import
java.sql.DriverManager; import
java.sql.SQLException; public class
DBConnection {    private static final
String URL =
    "jdbc:mysql://localhost:3306/smart_campus_parking";
private static final String USER = "root";    private static
final String PASSWORD = "your_password";    public static
Connection getConnection()        throws SQLException {
return DriverManager.getConnection(
    URL,
    USER,
    PASSWORD
);
}
}
}

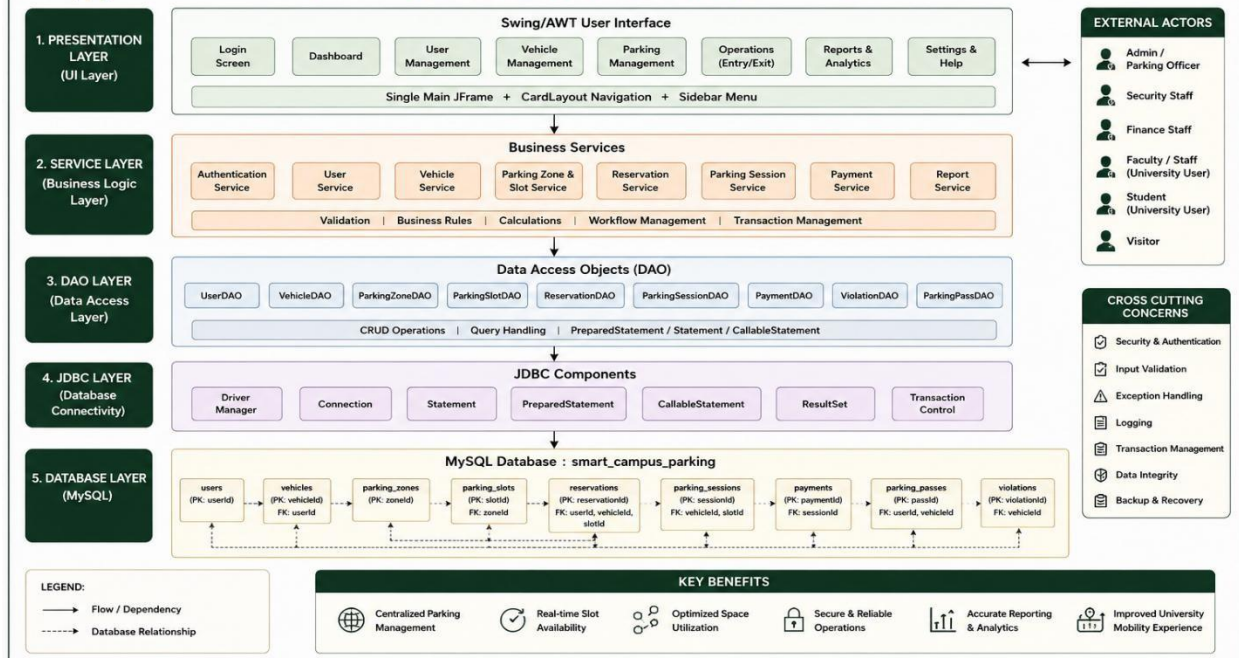
```



# AURELIA UNIVERSITY

## CAMPUS PARKING & MOBILITY OFFICE SYSTEM

### SYSTEM ARCHITECTURE



# AURELIA UNIVERSITY

## CAMPUS PARKING & MOBILITY OFFICE SYSTEM

### ER DIAGRAM

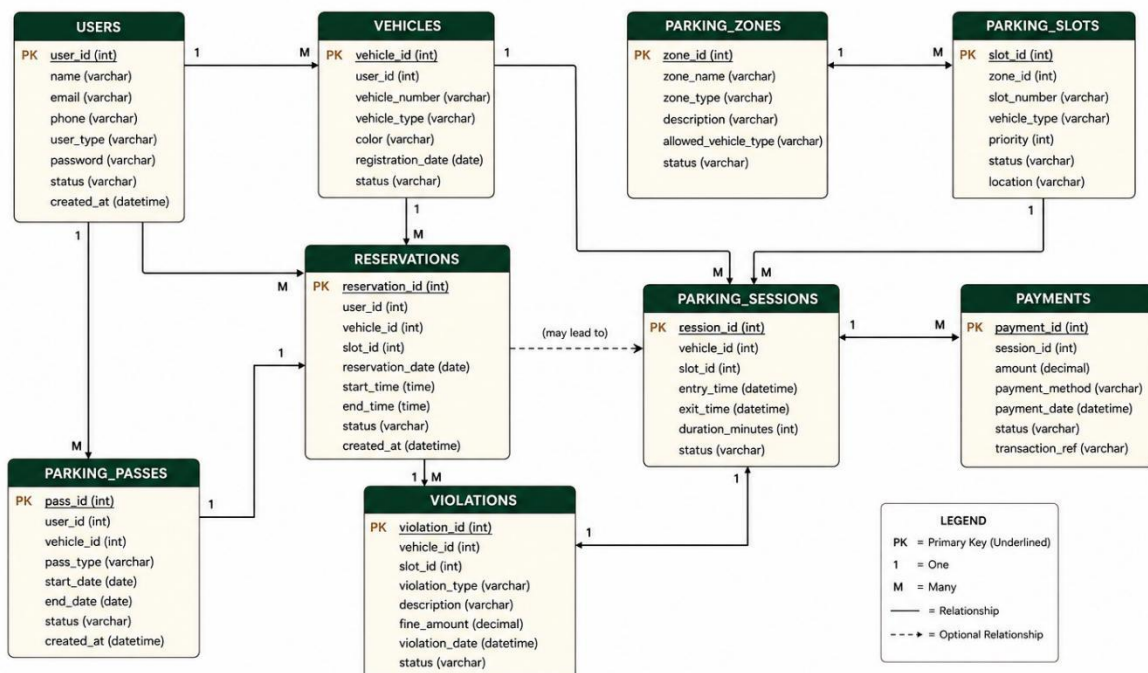
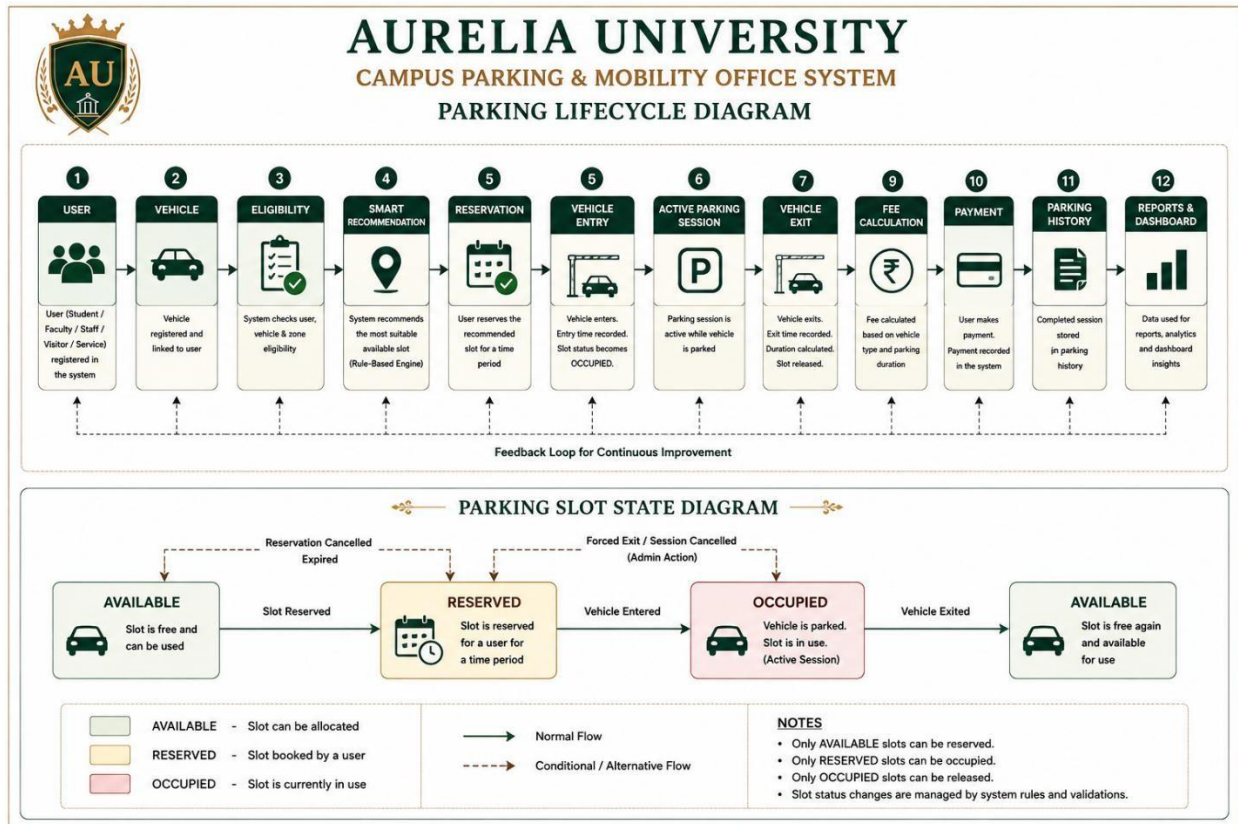


Figure 3.3 – Parking Lifecycle & Slot State Diagram



## 4. TOOLS & TECHNIQUES

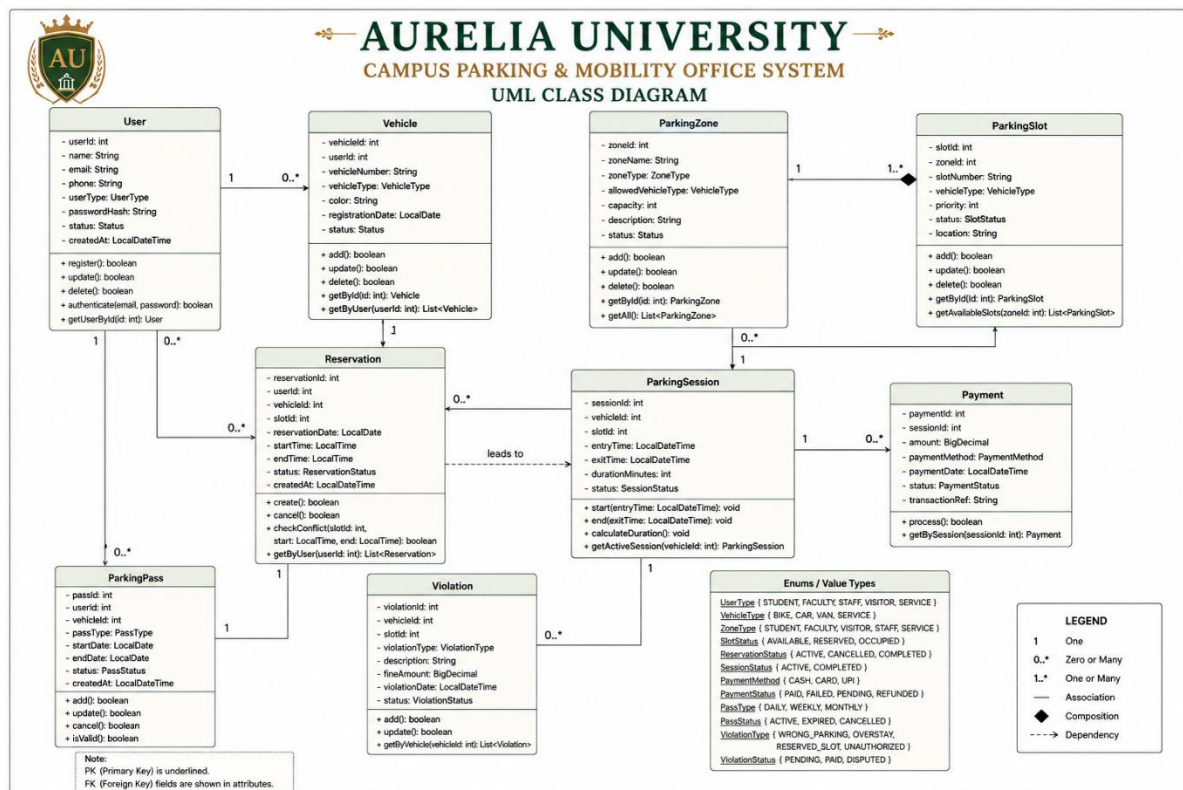
Table 4.1 – Technologies and Tools Used

| Technology / Tool | Purpose in Project                                                                                   |
|-------------------|------------------------------------------------------------------------------------------------------|
| Java              | Core language for the entire application (domain classes, UI, service, DAO).                         |
| Java Swing / AWT  | GUI components (JFrame, JPanel, JTable) and underlying event/layout classes for all sixteen screens. |
| JDBC              | API used by the DAO layer to connect to and query MySQL.                                             |
| MySQL / SQL       | Relational database and query language for all persistent data and reporting.                        |
| CardLayout        | Switches between module panels within a single JFrame via the sidebar.                               |

|                                       |                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------|
| PreparedStatement                     | Parameterised, injection-safe SQL for all user-input queries.                 |
| Statement                             | Used only for simple, static, read-only queries with no user input.           |
| CallableStatement / Stored Procedures | Invokes server-side routines for meaningful multi-step operations where used. |
| DAO Pattern / Service Layer           | Isolates SQL in DAO classes; centralises business rules in a service layer.   |

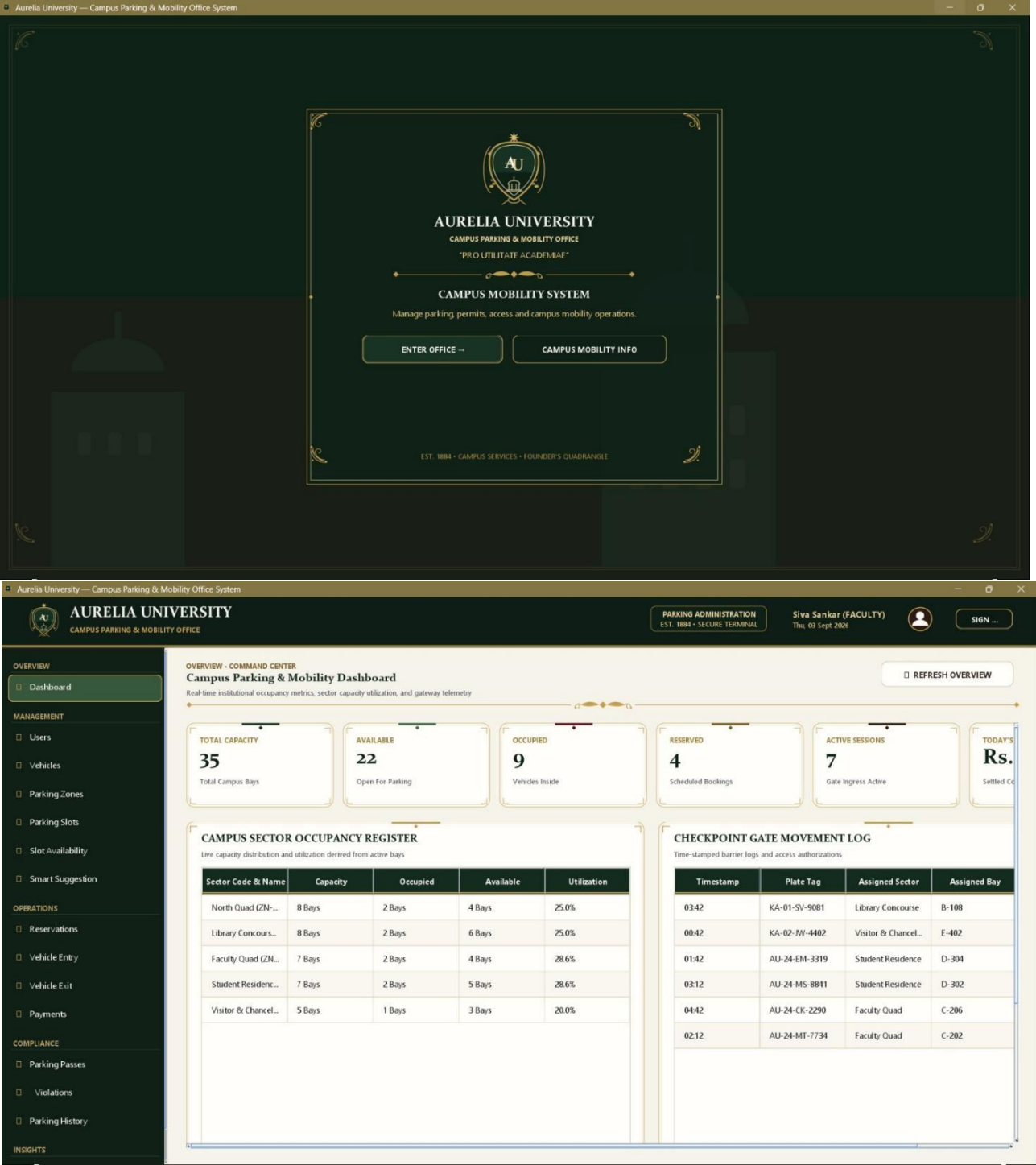
Swing/AWT is appropriate for this internal, staff-facing administrative tool because it needs no web server or browser-compatibility layer and ships with the standard JDK. JDBC is appropriate because it maps directly to the relational schema without an additional persistence framework. Prepared Statement is the default for any user-supplied value to prevent SQL injection; Statement is used only for static lookups; Callable Statement is reserved for genuine stored-procedure logic.

safeguard against data corruption even where application logic might contain a bug.

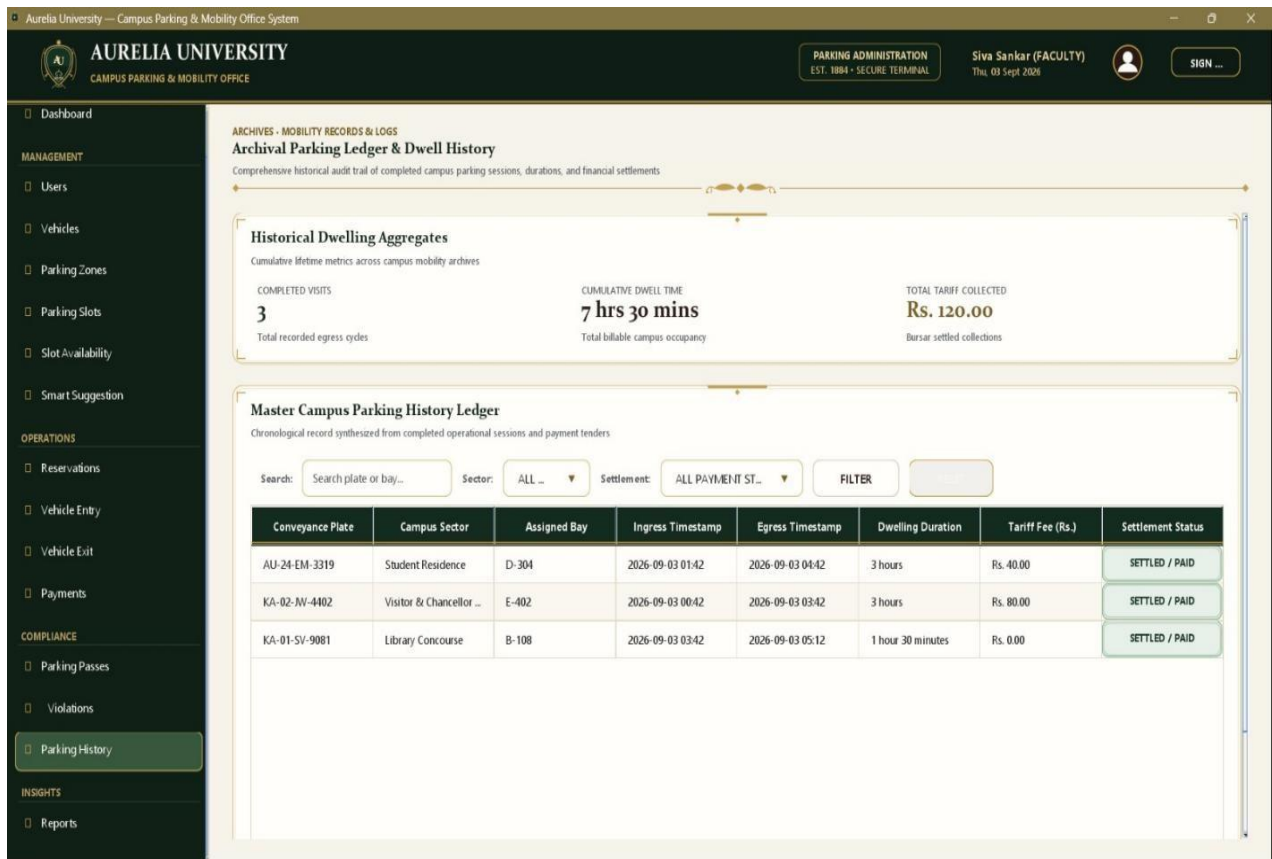


## 9.APPLICATION INTERFACE & SCREENSHORTS

Each of the sixteen module screens listed below requires an actual screenshot once the application has been built and run; none are fabricated here.



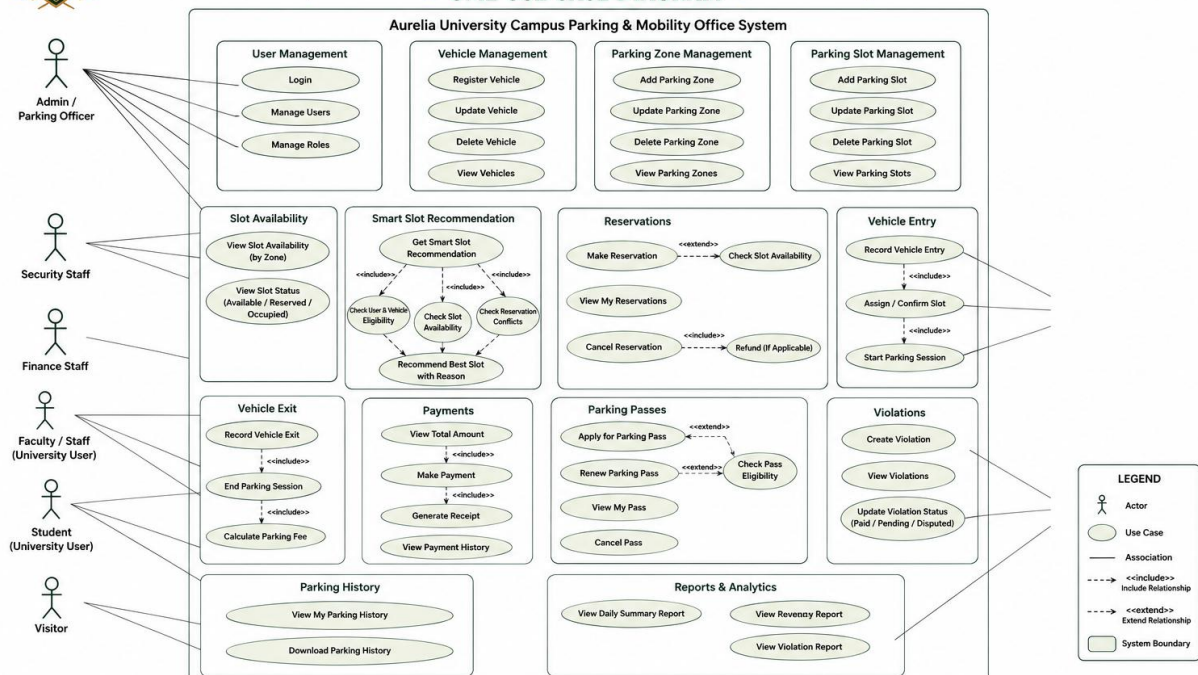




# AURELIA UNIVERSITY

## CAMPUS PARKING & MOBILITY OFFICE SYSTEM

### UML USE CASE DIAGRAM



## **Individual Contribution – Member 1**

**Madhu Sneha Latha Reddy – 192425092**

### **Frontend, User Interface & Parking Operations**

The first member was primarily responsible for the design and development of the application's frontend and parking-operation modules. The contribution included:

- Designed the overall Aurelia University – Old Money × University visual theme.
- Developed the Java Swing/AWT user interface and main application window.
- Implemented the single JFrame and CardLayout navigation architecture.
- Developed the sidebar navigation and dashboard interface.
- Designed reusable UI components such as buttons, text fields, tables, status badges, metric cards and parking-slot components.
- Developed the Login, Dashboard, User Management and Vehicle Registration interfaces.
- Implemented the Parking Zones, Parking Slots and Slot Availability modules.
- Developed the Smart Slot Recommendation interface and rule-based recommendation workflow.
- Worked on the Reservations, Vehicle Entry and Vehicle Exit modules.
- Designed the visual parking-slot grid and slot-status representation.
- Implemented frontend-side input validation and user interaction handling.
- Performed UI testing, navigation testing and screen-level debugging.
- Contributed to the preparation of UML diagrams, system architecture diagrams and application screenshots.
- Assisted in final integration, testing and documentation.

### **Main Responsibility**

Frontend development, user interaction, parking operations and application interface design.

## **Individual Contribution – Member 2**

**Asmitha Kadiri –192425038**

### **Database, Backend, JDBC & Business Logic**

The second member was primarily responsible for the database design, backend implementation and business logic of the system. The contribution included:

- Designed the MySQL database architecture for the campus parking system.
- Created and managed the smart\_campus\_parking database.
- Designed the major database tables:
  - users
  - vehicles
  - parking\_zones
  - parking\_slots
  - reservations
  - parking\_sessions
  - parking\_passes
  - payments
  - violations
- Defined primary keys, foreign keys, constraints and relationships between entities.
- Developed the JDBC database connectivity layer.
- Implemented the DAO layer for database operations.
- Implemented CRUD operations for appropriate entities.
- Used PreparedStatement for parameterized user-input queries.
- Used Statement for appropriate static/read-only queries.
- Used CallableStatement for stored-procedure execution where applicable.
- Developed and/or integrated the business/service layer.
- Implemented validation and business rules related to:
  - Duplicate vehicles
  - Slot availability

- Reservation conflicts
- Entry and exit validation
- Parking-session management
- Payment validation
- Parking-pass validity
- Implemented parking-slot state management:  
AVAILABLE → RESERVED → OCCUPIED → AVAILABLE
- Worked on parking-duration and fee calculation logic.
- Implemented transaction handling using commit and rollback for critical operations.
- Worked on database exception handling and integrity constraints.
- Prepared realistic sample data for testing.
- Performed backend and database testing.
- Contributed to the ER diagram, database documentation and technical report.
- Assisted with final system integration and debugging.

### **Main Responsibility**

Database development, JDBC connectivity, backend logic, business rules and data integrity.

Github Link: [https://github.com/ashmitha-kadiri/Java\\_Assignment](https://github.com/ashmitha-kadiri/Java_Assignment)

Github Link: [https://github.com/192425092/Java\\_Assignment](https://github.com/192425092/Java_Assignment)